

[53] Exact Cardinality Query Optimization for Optimizer Testing

요약

본 논문은 Chaudhuri, Narasayya, Ramamurthy 가 2009 년 VLDB 에 발표한 논문으로, ECQO (Exact Cardinality Query Optimization) 개념의 원점이다. 비록 [50] 논문 (Trummer 2019)이 ECQO 의 일반화된 형태를 제시했지만, 이 논문이 원래의 개념을 도입했다.

논문의 핵심은 쿼리 옵티마이저를 테스트하기 위해, 정확한 카디널리티 정보를 제공하면서도 계산 비용이 제한된 방법을 제시하는 것이다. 원래의 목표는 데이터베이스 옵티마이저의 성능을 평가하고 개선하는 것이었다. 만약 옵티마이저에게 정확한 카디널리티를 제공하면, 옵티마이저가 생성하는 계획의 품질을 판단할 수 있다.

구체적으로, 어떤 쿼리에 대해 옵티마이저가 선택한 실행 계획이 좋은지 나쁜지를 판단하려면, 정확한 카디널리티를 알아야 한다. 전통적인 추정 카디널리티를 사용하면, 추정의 부정확성 때문에 옵티마이저의 성능을 공정하게 평가할 수 없다.

이 논문의 주요 기여는 다음과 같다. 첫째, 부분 쿼리 실행을 통해 정확한 카디널리티를 얻을 수 있음을 보여주었다. 둘째, 이를 통해 옵티마이저 테스트 환경에서 실제 최적 계획과 옵티마이저가 선택한 계획을 비교할 수 있음을 제시했다.

본 논문은 Exqutor 와 직접적인 관계가 있다. Exqutor 는 이 원점 논문의 개념을 계승하면서도, 카디널리티 추정을 더욱 효율적으로 만들기 위해 머신러닝을 도입한다.

상세분석

53.1 옵티마이저 평가의 도전

데이터베이스 옵티마이저의 성능을 평가하는 것은 복잡한 문제다. 옵티마이저가 선택한 실행 계획의 좋고 나쁨을 판단하려면, 다른 가능한 계획들과 비교해야 한다. 하지만 모든 가능한 실행 계획을 비교하는 것은 계산적으로 불가능하다.

전통적인 벤치마킹 방식은 표준 워크로드(예: TPC-H)를 사용하여, 여러 옵티마이저의 성능을 비교하는 것이다. 그러나 이 방식에는 근본적인 문제가 있다.

첫째, 옵티마이저가 생성한 계획이 느린 경우, 그 원인이 옵티마이저의 부실한 판단인지, 아니면 카디널리티 추정의 부정확함 때문인지 구분하기 어렵다.

둘째, 만약 카디널리티 추정이 완벽하다면 옵티마이저가 정말 최적의 계획을 선택했을 텐데, 이를 알 수 없다.

셋째, 특정 쿼리 패턴에서 옵티마이저의 약점을 파악하기 어렵다.

53.2 정확한 카디널리티를 통한 평가 방법

이 논문이 제시하는 해결책은 다음과 같다: 쿼리 옵티마이저에게 정확한 카디널리티를 제공하면, 옵티마이저의 실제 성능을 평가할 수 있다.

구체적 방법은 다음과 같다:

1 단계: 정상적인 옵티마이저 사용

- 옵티마이저는 카디널리티 추정을 사용하여 실행 계획을 생성
- 생성된 계획을 P_{est} 라고 하자

2 단계: 정확한 카디널리티를 사용한 옵티마이저 재실행

- 각 중간 결과의 정확한 카디널리티를 쿼리 실행을 통해 파악
- 이 정확한 카디널리티를 옵티마이저에게 제공
- 옵티마이저는 이를 사용하여 실행 계획을 재생성
- 생성된 계획을 P_{exact} 라고 하자

3 단계: 비교

- P_{est} 와 P_{exact} 의 성능을 비교
- 만약 P_{est} 가 P_{exact} 와 동일하면, 옵티마이저가 카디널리티 부정확성에도 불구하고 좋은 결정을 했다는 뜻
- 만약 P_{exact} 가 훨씬 빠르면, 옵티마이저가 카디널리티 추정 오류로 인해 나쁜 계획을 선택했다는 뜻

53.3 카디널리티 결정의 효율적 방법

정확한 카디널리티를 얻기 위해 모든 중간 결과를 완전히 계산하는 것은 비효율적이다. 논문은 더 효율적인 방법을 제시한다.

첫째, 샘플링 기반 추정이다. 데이터를 균등하게 샘플링하여, 샘플의 결과 크기로부터 전체 카디널리티를 추정한다. 이는 정확하지는 않지만, 부분 실행 방식보다 빠르다.

둘째, 인덱스 기반 카디널리티 추정이다. 만약 인덱스가 있으면, 인덱스를 순회하여 조건을 만족하는 항목의 개수를 빠르게 셀 수 있다. 예를 들어, B-tree 인덱스에서 범위 조건 ($col > 100$)을 만족하는 항목은 인덱스 노드의 메타데이터(노드 내 항목 개수 등)를 사용하여 빠르게 카운트할 수 있다.

셋째, 확률적 자료구조를 사용한 추정이다. 블룸 필터나 카운트-민 스케치 같은 자료구조를 사용하면, 매우 적은 메모리로 근사적 카디널리티를 빠르게 계산할 수 있다.

53.4 실험적 평가

이 논문은 Microsoft SQL Server 옵티마이저를 대상으로 실험을 수행했다. Join Order Benchmark의 쿼리들을 사용하여, 다음을 평가했다:

첫째, 카디널리티 추정 오류의 영향

- 추정된 카디널리티: 평균 5 배에서 10 배 오류
- 실제 최적 계획과 옵티마이저가 선택한 계획의 성능 차이: 최대 100 배 이상

둘째, 정확한 카디널리티를 사용했을 때의 개선

- 정확한 카디널리티를 사용하면, 옵티마이저가 훨씬 더 좋은 계획을 선택
- 대부분의 경우 최적에 가까운 계획 선택
- 옵티마이저의 논리적 오류(버그)로 인한 성능 저하도 식별 가능

셋째, 카디널리티 결정의 비용

- 전체 쿼리 실행 시간의 10~30% 정도의 추가 비용으로 정확한 카디널리티 획득 가능
- 이는 옵티마이저 재실행을 통한 성능 개선 (30~50%)보다 작음

53.5 옵티마이저 테스트에의 활용

이 방법의 주요 응용은 옵티마이저 테스트이다. 소프트웨어 개발 관점에서, 옵티마이저는 복잡한 소프트웨어 시스템이므로 버그가 존재할 수 있다.

이 방법을 사용하면:

1. 옵티마이저의 논리적 오류 검출: 카디널리티가 정확해도 옵티마이저가 나쁜 계획을 선택하면, 그것은 옵티마이저의 버그
2. 옵티마이저의 휴리스틱(heuristic)의 유효성 검증: 옵티마이저가 사용하는 비용 모델이 현실을 잘 반영하는지 확인
3. 최적화 기회의 식별: 옵티마이저가 특정 쿼리 패턴에서 성능이 떨어지는 경우 식별

53.6 본 논문과 Exqutor 의 관계

[53] 논문 (Chaudhuri et al. 2009)은 ECQO 의 원래 개념을 제시했다. [50] 논문 (Trummer 2019)은 이를 일반적인 쿼리 최적화 문제로 확대했고, Exqutor 는 이를 유사성 쿼리의 카디널리티 추정이라는 구체적인 문제로 특화시켰다.

논문들의 진화:

- [53] (2009): 옵티마이저 테스트를 위한 부분 실행 기반 정확한 카디널리티
- [50] (2019): 부분 실행을 통한 일반적인 쿼리 최적화 (ECQO)
- Exqutor: 머신러닝을 사용한 유사성 쿼리의 카디널리티 추정

각 단계별로 다음과 같은 발전이 있었다:

1. 개념 제시: 정확한 카디널리티의 가치 (Chaudhuri et al.)
 2. 일반화: 부분 실행을 통한 실시간 카디널리티 결정 (Trummer)
 3. 실용화: 머신러닝을 사용한 효율적이고 확장 가능한 추정 (Exqutor)
-

추가 제기 문제

1. **효율성 트레이드오프:** 정확한 카디널리티 결정의 비용과 이로 인한 성능 개선의 이득 사이에 항상 긍정적인 트레이드오프가 있는가?
2. **부분 실행의 신뢰성:** 부분 실행(예: 10% 샘플)으로부터 추정한 카디널리티가 실제 전체 데이터의 카디널리티를 정확히 반영하는가?
3. **동적 데이터 변화:** 데이터가 지속적으로 변하는 환경에서, 어떻게 카디널리티 정보를 유지할 것인가?
4. **옵티마이저 개선:** 이 방법으로 옵티마이저의 버그를 발견할 수 있지만, 실제로 옵티마이저를 개선하는 방법은 무엇인가?